

Weekly Progress Report

Name:Joshika Crisane J

Domain: Data Science and machine learning

Date of Submission:23/6/26

Week Ending: 01

I. Overview

This week, the primary focus was on understanding the USC_TIA learning platform while developing a Machine Learning project on **Forecasting of Smart City Traffic Patterns**. The project involved data preprocessing, feature engineering, model development using Random Forest Regression, prediction of traffic volume, and visualization of results.

II. Achievements

1. USC_TIA Familiarization

- Explored the USC_TIA platform and understood its learning modules and project workflow.
- Utilized the platform to strengthen knowledge of Python programming and Machine Learning.
- Successfully completed the assigned learning activities and practical exercises.

2. Python Project Contributions

Name of the Project: Forecasting of Smart City Traffic Patterns

- Imported the traffic dataset and explored its structure.
- Checked the dataset for missing values and removed duplicate records.
- Converted the date and time columns into datetime format.
- Extracted useful features such as Year, Month, Day, Hour, Day of Week, and Is Weekend from the datetime column.
- Performed data preprocessing and feature selection for model training.
- Split the dataset into training and testing sets.
- Built a Random Forest Regression model to predict traffic volume.
- Evaluated the model using appropriate regression metrics.
- Visualized the actual and predicted traffic patterns using graphical plots.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

train_data = pd.read_csv("/content/drive/MyDrive/train_awnotu8.csv")
test_data = pd.read_csv("/content/drive/MyDrive/datasets_8494_11879_test_BdBKkAj.csv")

train_data.head()
Show hidden output

train_data.head()
Show hidden output

train_data.isnull().sum()
Show hidden output

test_data.isnull().sum()
Show hidden output

train_data.duplicated().sum()

np.int64(0)

plt.hist(train_data["Vehicles"], bins=20,color="orange")
plt.xlabel("Vehicles")
plt.ylabel("Frequency")
plt.title("Distribution of Vehicle Count")
plt.show()
Show hidden output

train_data.groupby("Junction")["Vehicles"].mean().plot(kind="bar",color='pink')
plt.xlabel("Junction")
plt.ylabel("Average Vehicles")
plt.title("Average Traffic at Each Junction")
plt.show()

train_data.groupby("Junction")["Vehicles"].mean().plot(kind="bar",color='pink')
plt.xlabel("Junction")
plt.ylabel("Average Vehicles")
plt.title("Average Traffic at Each Junction")
plt.show()
Show hidden output

test_ids = test_data["ID"]

train_data.drop("ID", axis=1, inplace=True)

test_data.drop("ID", axis=1, inplace=True)

train_data["DateTime"] = pd.to_datetime(train_data["DateTime"])

test_data["DateTime"] = pd.to_datetime(test_data["DateTime"])

train_data["Year"] = train_data["DateTime"].dt.year
train_data["Month"] = train_data["DateTime"].dt.month
train_data["Day"] = train_data["DateTime"].dt.day
train_data["Hour"] = train_data["DateTime"].dt.hour
train_data["DayOfWeek"] = train_data["DateTime"].dt.dayofweek
train_data["Weekend"] = train_data["DayOfWeek"].isin([5,6]).astype(int)

test_data["Year"] = test_data["DateTime"].dt.year
test_data["Month"] = test_data["DateTime"].dt.month
test_data["Day"] = test_data["DateTime"].dt.day
test_data["Hour"] = test_data["DateTime"].dt.hour
test_data["DayOfWeek"] = test_data["DateTime"].dt.dayofweek
test_data["Weekend"] = test_data["DayOfWeek"].isin([5,6]).astype(int)

train_data.drop("DateTime", axis=1, inplace=True)

test_data.drop("DateTime", axis=1, inplace=True)

X_train = train_data.drop("Vehicles", axis=1)

y_train = train_data["Vehicles"]

X_test = test_data

model = RandomForestRegressor(random_state=42)

model.fit(X_train, y_train)

RandomForestRegressor
RandomForestRegressor(random_state=42)

predictions = model.predict(X_test)

plt.hist(predictions, bins=20,color="green")
plt.xlabel("Vehicles")
plt.ylabel("Frequency")
plt.title("Distribution of Vehicle Count")
plt.show()
... Show hidden output
```

3. Learning Python

- Learned how to preprocess time-series traffic data using Pandas.
- Understood feature engineering techniques for datetime attributes.
- Gained practical experience in implementing Random Forest Regression using Scikit-learn.
- Improved skills in data visualization using Matplotlib.

III. Challenges

1. USC_TIA Integration

- Faced initial challenges in understanding how the USC_TIA learning modules relate to practical Machine Learning projects.
- Continued exploring platform resources to strengthen conceptual understanding.

2. Python Project Complexity

- Encountered difficulty while converting date and time columns into meaningful features.
- Learned the importance of feature engineering in improving model performance.
- Gained a better understanding of Random Forest Regression and its application to traffic prediction.

IV. Learning Resources

1. USC_TIA Documentation

- Referred to USC_TIA learning materials to understand project objectives and implementation steps.
- Utilized platform resources to reinforce Python and Machine Learning concepts.

2. Python Learning Resources

- Used Pandas documentation for data preprocessing and datetime operations.
- Studied Scikit-learn documentation to implement Random Forest Regression.
- Practiced Matplotlib for data visualization.
- Referred to online tutorials to understand traffic forecasting and feature engineering techniques.

V. Next Week's Goals

1. USC_TIA Enhancement

- Continue exploring advanced USC_TIA learning modules and complete additional practical exercises.
- Apply newly learned Machine Learning concepts to real-world datasets.

2. Python Project Development

- Improve the prediction accuracy by tuning Random Forest model parameters.
- Explore additional regression algorithms for comparison.
- Enhance visualization techniques to better interpret traffic forecasting results.

VI. Additional Comments

This week's project provided hands-on experience in building a complete Machine Learning workflow for traffic forecasting. The project enhanced my understanding of data preprocessing, datetime feature engineering, Random Forest Regression, model evaluation, and visualization. The USC_TIA platform supported both conceptual learning and practical implementation throughout the project.

Weekly Progress Report

Name:J.Joshika Crisane

Domain: Data Science and machine learning

Date of Submission:23/6/26

Week Ending: 01

I. Overview

This week, the primary focus was on understanding the USC_TIA learning environment while applying Python and Machine Learning concepts through a Crop Yield Prediction project. The work involved data preprocessing, exploratory data analysis (EDA), outlier detection, model development using Linear Regression, and visualization of prediction results.

II. Achievements

1. USC_TIA Familiarization

- Explored the USC_TIA platform and understood its learning objectives and workflow.
- Utilized the platform to enhance Python programming and Machine Learning skills.
- Successfully completed assigned learning activities and practical exercises.

2. Python Project Contributions

Name of the Project: Crop Yield Prediction using Linear Regression

- Imported and analyzed the crop yield dataset.
- Performed data cleaning and checked for missing values.
- Conducted Exploratory Data Analysis (EDA) using statistical summaries and visualizations.
- Detected and removed outliers using the Interquartile Range (IQR) method.
- Built a Linear Regression model for crop yield prediction.
- Evaluated the model using Mean Absolute Error (MAE) and Mean Squared Error (MSE).
- Created scatter plots to compare actual and predicted crop yields.

Code:

```
1) ❶ import pandas as pd
ts import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```
1) data = pd.read_csv("/content/datafile (1).csv")
Os
```

```
1) data.head()
Os
```

Show hidden output

Next steps: [Generate code with data](#) [New interactive sheet](#)

```
1) data.tail()
Os
```

Show hidden output

```
1) data.info()
Os
```

Show hidden output

```
1) data.shape
Os
```

Show hidden output

```
1) data.describe()
Os
```

Show hidden output

```
1) data.isnull().sum()
Os
```

```
data.duplicated().sum()
```

Show hidden output

```
data.hist(figsize=(10,8),color="green")
```

```
plt.show()
```

Show hidden output

```
data["Crop"].value_counts().plot(
    kind="bar",
    color="green",
    figsize=(8,5)
)
```

```
plt.title("Crop Distribution")
plt.xlabel("Crop")
plt.ylabel("Count")
plt.show()
```

Show hidden output

```
data["State"].value_counts().plot(
    kind="bar",
    color="red",
    figsize=(10,5)
)
```

```
plt.title("State Distribution")
plt.xlabel("State")
plt.ylabel("Count")
plt.show()
```

Show hidden output

```
Q1 = data.quantile(0.25)
Q3 = data.quantile(0.75)
IQR = Q3 - Q1
```

```
data = data[~((data < (Q1 - 1.5 * IQR)) | (data > (Q3 + 1.5 * IQR))).any(axis=1)]
```

```
data.boxplot(figsize=(10,6))
```

Show hidden output

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
data["Crop"] = le.fit_transform(data["Crop"])
```

```
data["State"] = le.fit_transform(data["State"])
```

```
X = data.drop("Yield (Quintal/ Hectare) ", axis=1)
```

```
y = data["Yield (Quintal/ Hectare) "]
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
X_train
```

Show hidden output

Next steps: [Generate code with X_train](#) [New interactive sheet](#)

```
❶ from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

Show hidden output

```
y_pred = model.predict(X_test)
```

```
❶ print("MAE:",mean_absolute_error(y_test, y_pred))
print("MSE:",mean_squared_error(y_test, y_pred))
```

```
MAE: 5.934252014992069
MSE: 44.90846137404369
```

```
plt.figure(figsize=(8,6))
```

```
plt.scatter(y_test, y_pred)
```

```
plt.xlabel("Actual Yield")
plt.ylabel("Predicted Yield")
plt.title("Actual vs Predicted Yield")
```

```
plt.grid(True)
plt.show()
```

Show hidden output

3. Learning Python

- Improved understanding of data preprocessing and feature analysis.
- Learned how to implement Linear Regression using Scikit-learn.
- Gained hands-on experience with Python libraries including Pandas, NumPy, Matplotlib, and Scikit-learn.
- Understood the importance of outlier detection and model evaluation in Machine Learning.

III. Challenges

1. USC_TIA Integration

- Faced initial difficulty understanding how the learning modules connect with practical Machine Learning projects.
- Continued exploring USC_TIA resources to strengthen conceptual understanding.

2. Python Project Complexity

- Initially found the IQR-based outlier removal technique and boolean indexing challenging.
- Improved understanding of regression evaluation metrics and prediction visualization through practice.

IV. Learning Resources

1. USC_TIA Documentation

- Referred to USC_TIA learning materials and documentation to understand project requirements.
- Utilized platform resources to reinforce Python and Machine Learning concepts.

2. Python Learning Resources

- Studied Scikit-learn documentation for implementing Linear Regression.
- Used Pandas documentation for data manipulation and preprocessing.
- Practiced Matplotlib for data visualization and result analysis.
- Referred to online tutorials to strengthen knowledge of EDA and outlier detection techniques.

V. Next Week's Goals

1. USC_TIA Enhancement

- Continue exploring advanced USC_TIA learning modules.
- Apply newly learned concepts to additional Machine Learning tasks and assignments.

2. Python Project Development

- Improve model performance by experimenting with additional preprocessing techniques.
- Compare Linear Regression with other regression algorithms.
- Enhance data visualization and model evaluation skills through further practice.

VI. Additional Comments

This week's activities provided practical exposure to the complete Machine Learning workflow, including data preprocessing, exploratory analysis, model development, evaluation, and visualization. The USC_TIA learning platform helped strengthen both theoretical understanding and hands-on implementation of Python and Machine Learning concepts.